

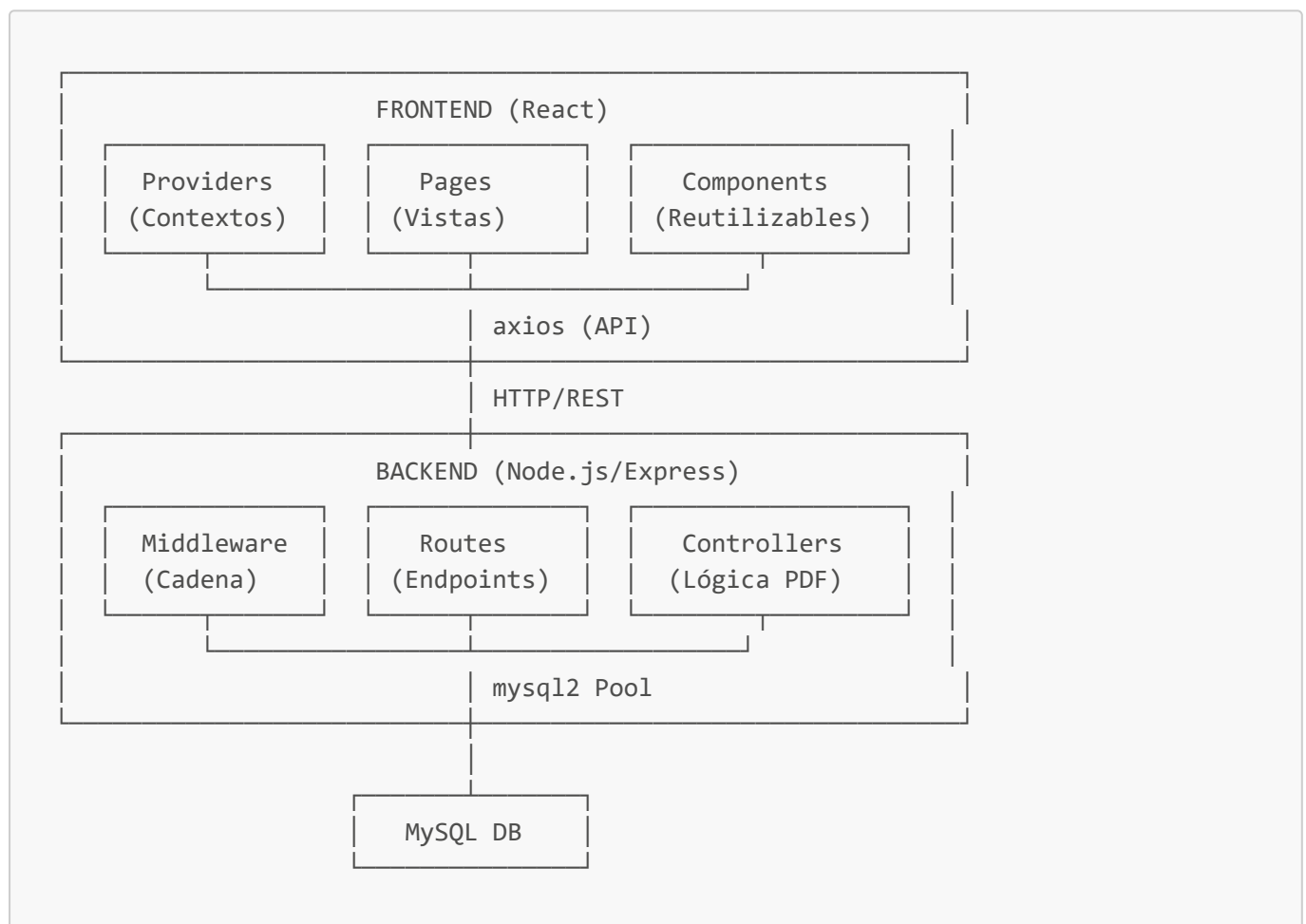
DOC_POO.md — Programación Orientada a Objetos

Proyecto ANA-VET · Sistema SaaS Veterinario

1. Introducción

ANA-VET es una aplicación full-stack que, aunque utiliza JavaScript (un lenguaje multiparadigma), aplica de forma consistente los principios de la **Programación Orientada a Objetos** tanto en el backend (Node.js/Express) como en el frontend (React). Este documento analiza cómo se manifiestan los conceptos de POO en el código real del proyecto.

2. Arquitectura General del Sistema



3. Principios de POO Aplicados

3.1 Encapsulamiento

El encapsulamiento consiste en ocultar los detalles internos de implementación y exponer solo lo necesario.

Backend — Módulo de Conexión ([src/db/connection.js](#))

```
// Encapsula la configuración del pool de conexiones
const db = mysql.createPool({
  host: process.env.DB_HOST,    // Detalles internos ocultos
  user: process.env.DB_USER,
  password: process.env.DB_PASS,
  database: process.env.DB_NAME,
  connectionLimit: 10,
});

// Expone solo la interfaz pública necesaria
function query(sql, params) {
  return new Promise((resolve, reject) => {
    db.query(sql, params, (err, rows) => {
      if (err) return reject(err);
      resolve(rows);
    });
  });
}

module.exports = { db, query }; // Solo se exporta lo necesario
```

Análisis: El pool de conexiones (`db`) y su configuración interna están encapsulados. Los módulos consumidores solo necesitan llamar a `query(sql, params)` sin conocer los detalles del pool.

Backend — Módulo de Autenticación (`src/middleware/authMiddleware.js`)

```
// Constantes internas encapsuladas
const JWT_SECRET = process.env.JWT_SECRET || 'saas_vet_secret_2026';
const SALT_ROUNDS = 10;

// Funciones con responsabilidad única, encapsuladas
function authMiddleware(req, res, next) { /* ... */ }
function soloClinica(req, res, next) { /* ... */ }
function soloEmpleado(req, res, next) { /* ... */ }
function clinicaOVeterinario(req, res, next) { /* ... */ }

// Interfaz pública controlada
module.exports = { authMiddleware, soloClinica, soloEmpleado, clinicaOVeterinario,
  JWT_SECRET, SALT_ROUNDS };
```

Frontend — AuthContext (`src/context/AuthContext.js`)

```
// Estado interno encapsulado
const SESSION_KEY = 'ana_vet_session';

// Funciones privadas (no exportadas)
function loadSession() { /* ... */ }
```

```
function persistSession(session) { /* ... */ }

// Solo se expone la interfaz pública a través del contexto
const value = {
  token, tipo, rol, clinica_id, user, isLoggedIn, // Estado
  loginClinica, loginEmpleado, logout,          // Acciones
};
```

3.2 Abstracción

La abstracción permite trabajar con conceptos de alto nivel sin preocuparse por los detalles de implementación.

Backend — Abstracción de Queries con Promise Wrapper

```
// Abstracción: el consumidor no sabe si es callback o promise internamente
const { query } = require('../db/connection');

// Uso en dashboard.js – alto nivel, sin callbacks
const [rowsIngresos, rowsPacientes, rowsConsultasHoy] = await Promise.all([
  query(sqlIngresos, [clinica_id]),
  query(sqlPacientes, [clinica_id]),
  query(sqlConsultasHoy, [clinica_id]),
]);
```

Frontend — Abstracción de la Capa HTTP (src/api.js)

```
// Abstracción: todos los componentes usan API sin conocer la URL base
const API = axios.create({
  baseURL: process.env.REACT_APP_API_URL || 'http://localhost:4000/api',
  headers: { 'Content-Type': 'application/json' },
});

// Uso en cualquier componente:
API.get('/dashboard/clinica') // Sin conocer la URL completa
API.post('/clinicas/login', {}) // Sin gestionar headers manualmente
```

Frontend — Abstracción de Colores de Animales (SelectedAnimalContext.js)

```
// Abstracción: el consumidor solo llama getAnimalColor(animal)
// sin conocer la lógica de mapas de colores por especie/raza
function getAnimalColor(animal) {
  const especie = animal.especie.toLowerCase();
  const breed = animal.raza?.trim().toLowerCase();
  if (breed) {
```

```

    return breedColorMap[breed] ?? stringToColor(breed);
  }
  return speciesColorMap[especie] ?? speciesColorMap.default;
}

```

3.3 Herencia (Composición sobre Herencia)

JavaScript moderno favorece la **composición** sobre la herencia clásica. ANA-VET aplica este principio extensamente.

Frontend — Composición de Providers (App.js)

```

// Composición de múltiples contextos/providers
function App() {
  return (
    <SelectedAnimalProvider>    {/* Contexto de animal seleccionado */}
      <ThemeProvider>          {/* Contexto de tema oscuro/claro */}
        <AuthProvider>         {/* Contexto de autenticación */}
          <BrowserRouter>
            <AppRouter />
          </BrowserRouter>
        </AuthProvider>
      </ThemeProvider>
    </SelectedAnimalProvider>
  );
}

```

Cada Provider "hereda" (compone) el comportamiento del anterior, formando una cadena de responsabilidades.

Frontend — Composición de Componentes en Dashboard

```

// KpiCard es un componente base reutilizable
function KpiCard({ icon, label, value, sub, colorClass, bgClass, accent }) { /* ... */ }

// TareaCard es una variante con comportamiento extendido (statusDot)
function TareaCard({ icon, label, value, sub, colorClass, bgClass, borderClass, statusDot }) { /* ... */ }

// ActivityBadge encapsula lógica de presentación de actividad
function ActivityBadge({ count }) { /* ... */ }

// EmptyState es un componente de estado vacío reutilizable
function EmptyState({ emoji, title, subtitle }) { /* ... */ }

// Dashboard compone todos estos componentes

```

```
function DashboardClinica({ user }) {  
  return (  
    <>  
    <KpiCard ... />  
    <TareaCard ... />  
    <ActivityBadge ... />  
    <EmptyState ... />  
  </>  
);  
}
```

Backend — Composición de Middlewares (Cadena de Responsabilidad)

```
// Cada middleware es un objeto función con responsabilidad única  
// Se componen en cadena para construir comportamientos complejos  
router.get('/perfil',  
  authMiddleware,    // 1. Verificar token JWT  
  soloClinica,       // 2. Verificar tipo de usuario  
  (req, res) => { /* 3. Lógica de negocio */ }  
);  
  
// Composición más compleja  
router.post('/',  
  authMiddleware,    // Autenticación  
  soloClinica,       // Autorización de rol  
  async (req, res) => { /* Lógica */ }  
);
```

3.4 Polimorfismo

El polimorfismo permite que diferentes objetos respondan al mismo mensaje de formas distintas.

Frontend — Dashboard Polimórfico

```
// El mismo componente Dashboard se comporta diferente según el tipo de usuario  
export default function Dashboard() {  
  const { tipo, user } = useAuth();  
  
  if (tipo === 'clinica') {  
    return <DashboardClinica user={user} />; // Vista administrativa  
  }  
  
  if (tipo === 'empleado') {  
    return <DashboardEmpleado user={user} />; // Vista operativa  
  }  
}
```

```
    return <FallbackView />; // Vista de error
}
```

Análisis: El mismo componente **Dashboard** produce salidas completamente diferentes según el tipo de usuario, sin que el consumidor necesite conocer la implementación interna.

Frontend — Polimorfismo en Notificaciones

```
// El mismo objeto notificación se renderiza diferente según su tipo
const TIPO_CONFIG = {
  vacuna: { icon: '💉', color: 'bg-violet-100...', badge: 'Vacuna' },
  consulta: { icon: '📄', color: 'bg-teal-100...', badge: 'Consulta' },
};

// Polimorfismo: misma interfaz, diferente comportamiento
const cfg = TIPO_CONFIG[n.tipo] || TIPO_CONFIG.consulta;
// cfg.icon, cfg.color, cfg.badge se usan igual sin importar el tipo
```

Backend — Polimorfismo en Middlewares de Autorización

```
// Todos los middlewares tienen la misma firma (req, res, next)
// pero comportamiento diferente
function soloClinica(req, res, next) {
  if (req.user.tipo !== 'clinica') return res.status(403).json({...});
  next();
}

function soloEmpleado(req, res, next) {
  if (req.user.tipo !== 'empleado') return res.status(403).json({...});
  next();
}

function clinicaOVeterinario(req, res, next) {
  const { tipo, rol_id } = req.user;
  if (tipo === 'clinica' || (tipo === 'empleado' && Number(rol_id) === 2)) {
    return next();
  }
  return res.status(403).json({...});
}
// Todos se usan de la misma forma: router.get('/', authMiddleware, soloClinica, handler)
```

Backend — Polimorfismo en Generación de PDFs

```
// Todos los reportes tienen la misma interfaz (req, res)
// pero generan documentos completamente diferentes
```

```
exports.reportePacientes      = (req, res) => { /* PDF de pacientes */ };
exports.reporteTutores        = (req, res) => { /* PDF de tutores */ };
exports.reporteHospitalizaciones = (req, res) => { /* PDF de hospitalizaciones */
};
exports.reporteCirugias        = (req, res) => { /* PDF de cirugías */ };
exports.reporteConsultas       = (req, res) => { /* PDF de consultas */ };
exports.reporteVacunas          = (req, res) => { /* PDF de vacunas */ };
exports.reporteGeneral          = (req, res) => { /* PDF resumen ejecutivo */ };
exports.reporteExpediente       = (req, res) => { /* PDF de expediente individual */
};
```

4. Patrones de Diseño Implementados

4.1 Patrón Singleton — Conexión a Base de Datos

```
// src/db/connection.js
// El pool de conexiones es un Singleton: se crea una sola vez
// y se reutiliza en toda la aplicación
const db = mysql.createPool({ /* config */ });

// Todos los módulos importan la misma instancia
const { db } = require('../db/connection');
```

Beneficio: Evita crear múltiples pools de conexiones, optimizando el uso de recursos.

4.2 Patrón Observer — React Context + useEffect

```
// AuthContext.js – El estado de sesión es el "sujeto"
const [session, setSession] = useState(() => loadSession());

// useEffect actúa como "observador" que reacciona a cambios
useEffect(() => {
  persistSession(session); // Se ejecuta cada vez que session cambia
}, [session]);

// NotificacionesBell.js – Polling como Observer
useEffect(() => {
  fetchNotificaciones();
  const interval = setInterval(fetchNotificaciones, 5 * 60 * 1000);
  return () => clearInterval(interval); // Cleanup (unsubscribe)
}, [fetchNotificaciones]);
```

4.3 Patrón Strategy — Generación de Correos Únicos

```
// empleados.js – Estrategia de generación de email
// Estrategia 1: Email proporcionado manualmente
if (!generar_correo) {
  // Verificar unicidad y usar el email del body
}

// Estrategia 2: Email generado automáticamente
if (generar_correo) {
  const baseCorreo = generarBaseCorreo(nombre, apellidos, clinica_id);
  resolverCorreoUnico(baseCorreo, clinica_id, callback);
}

// Función de resolución de conflictos (parte de la estrategia)
function resolverCorreoUnico(baseCorreo, clinica_id, callback) {
  // Busca conflictos y agrega sufijo numérico si es necesario
  let contador = 1;
  let candidato = `${localBase}${contador}${dominio}`;
  while (existentes.includes(candidato)) { contador++; }
  return callback(null, candidato);
}
```

4.4 Patrón Factory — Creación de Sesiones

```
// AuthContext.js – Factory de objetos de sesión
const loginClinica = useCallback(async (email, password) => {
  const { data } = await API.post('/clinicas/login', { email, password });

  // Factory: crea un objeto de sesión estandarizado
  const newSession = {
    token: data.token,
    tipo: 'clinica',
    clinica_id: data.clinica.id,
    rol: 'Administrador',
    user: { id, nombre, email, telefono, direccion, logo_url },
  };
  setSession(newSession);
}, []);

const loginEmpleado = useCallback(async (email, password) => {
  const { data } = await API.post('/empleados/login', { email, password });

  // Factory: misma estructura, diferente contenido
  const newSession = {
    token: data.token,
    tipo: 'empleado',
    clinica_id: data.empleado.clinica_id,
    rol: data.empleado.rol_nombre,
    user: { id, nombre, apellidos, email, rol_id, rol_nombre, clinica_nombre },
  };
}, []);
```



```
    setSession(newSession);
  }, []);
```

4.5 Patrón Middleware (Chain of Responsibility)

```
// index.js – Cadena de middlewares global
app.use(cors());           // 1. CORS
app.use(express.json());   // 2. Parseo de JSON

// Rutas con cadena de responsabilidad
app.use('/api/clinicas', require('./src/routes/clinicas'));
// ...

app.use(errorHandler);     // Último eslabón: manejo de errores

// Por ruta – cadena específica
router.get('/clinica',
  soloClinica,             // 1. Verificar rol
  async (req, res) => {    // 2. Lógica de negocio
    // ...
  }
);
```

4.6 Patrón Template Method — Generación de PDFs

```
// reportsController.js – Método plantilla para todos los reportes
const addHeader = (doc, title) => {
  // Paso 1: Logo
  doc.image(LOGO_PATH, 50, 45, { width: 60 });
  // Paso 2: Nombre corporativo
  doc.text('ANA-vet', 120, 48);
  // Paso 3: Título del reporte
  doc.text(title, 50, 115, { align: 'center' });
  // Paso 4: Línea separadora
  doc.moveTo(50, 138).lineTo(550, 138).stroke();
};

const addTable = (doc, columns, rows, options = {}) => {
  // Paso 1: Encabezados
  // Paso 2: Filas con fondo alternado
  // Paso 3: Retornar posición Y final
};

// Cada reporte usa el mismo template
exports.reportePacientes = (req, res) => {
  db.query(query, [clinicaId], (err, rows) => {
    const doc = new PDFDocument();
```

```

    addHeader(doc, 'REPORTE DE PACIENTES'); // Template
    addTable(doc, columns, dataRows, opts); // Template
    doc.end();
  });
};

```

4.7 Patrón Provider (Context API como Dependency Injection)

```

// Los Providers inyectan dependencias en el árbol de componentes
// sin necesidad de prop drilling

// Proveedor
export function AuthProvider({ children }) {
  const value = { token, tipo, user, loginClinica, loginEmpleado, logout };
  return <AuthContext.Provider value={value}>{children}</AuthContext.Provider>;
}

// Consumidor – recibe las dependencias inyectadas
export function useAuth() {
  const ctx = useContext(AuthContext);
  if (!ctx) throw new Error('useAuth debe usarse dentro de <AuthProvider>');
  return ctx;
}

// Uso en cualquier componente profundo del árbol
function Sidebar() {
  const { tipo, logout } = useAuth(); // Inyección de dependencias
}

```

5. Componentes React como Objetos

En React, los componentes funcionales con hooks son equivalentes a objetos con estado y comportamiento:

5.1 Tabla de Equivalencias POO ↔ React

Concepto POO	Equivalente React
Clase	Componente funcional
Constructor	<code>useState(() => initialValue)</code>
Atributos de instancia	Variables de estado (<code>useState</code>)
Métodos	Funciones definidas dentro del componente
Métodos de ciclo de vida	<code>useEffect</code>
Herencia	Composición de componentes

Concepto POO	Equivalente React
Interfaz	Props (contrato de entrada)
Encapsulamiento	Closures + módulos ES6

5.2 Ejemplo: `NotificacionesBell` como Objeto

```
// Equivalente a una clase con:
export default function NotificacionesBell() {
  // Atributos de instancia (estado)
  const [notificaciones, setNotificaciones] = useState([]);
  const [open, setOpen] = useState(false);
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState(null);
  const dropdownRef = useRef(null); // Referencia al DOM

  // Método: fetchNotificaciones (con memoización)
  const fetchNotificaciones = useCallback(async () => {
    setLoading(true);
    try {
      const { data } = await API.get('/notificaciones');
      setNotificaciones(Array.isArray(data) ? data : []);
    } catch (err) {
      setError('No se pudieron cargar las notificaciones');
    } finally {
      setLoading(false);
    }
  }, []);

  // Ciclo de vida: componentDidMount + componentWillUnmount
  useEffect(() => {
    fetchNotificaciones();
    const interval = setInterval(fetchNotificaciones, 5 * 60 * 1000);
    return () => clearInterval(interval); // Destructor
  }, [fetchNotificaciones]);

  // Ciclo de vida: reacción a cambio de estado (open)
  useEffect(() => {
    function handleClickOutside(e) { /* ... */ }
    if (open) document.addEventListener('mousedown', handleClickOutside);
    return () => document.removeEventListener('mousedown', handleClickOutside);
  }, [open]);

  // Método render (retorno JSX)
  return ( /* JSX */ );
}
```

5.3 Ejemplo: `AuthProvider` como Objeto Singleton de Sesión

```

export function AuthProvider({ children }) {
  // Estado privado (encapsulado)
  const [session, setSession] = useState(() => {
    const saved = loadSession();
    if (saved?.token) {
      API.defaults.headers.common['Authorization'] = `Bearer ${saved.token}`;
    }
    return saved;
  });

  // Observador de cambios de estado
  useEffect(() => { persistSession(session); }, [session]);

  // Métodos públicos (memoizados para evitar re-renders)
  const loginClinica = useCallback(async (email, password) => { /* ... */ }, []);
  const loginEmpleado = useCallback(async (email, password) => { /* ... */ }, []);
  const logout = useCallback(() => { /* ... */ }, []);

  // Interfaz pública (equivalente a métodos públicos de una clase)
  const value = { token, tipo, rol, clinica_id, user, isLoggedIn,
    loginClinica, loginEmpleado, logout };

  return <AuthContext.Provider value={value}>{children}</AuthContext.Provider>;
}

```

6. Separación de Responsabilidades

6.1 Backend — Capas de la Aplicación

Capa	Archivo(s)	Responsabilidad
Configuración	<code>index.js</code>	Registro de rutas, middlewares globales, arranque del servidor
Conexión BD	<code>src/db/connection.js</code>	Pool de conexiones, wrapper Promise
Autenticación	<code>src/middleware/authMiddleware.js</code>	Verificación JWT, control de roles
Manejo de Errores	<code>src/middleware/errorHandler.js</code>	Respuestas de error uniformes
Rutas/Controladores	<code>src/routes/*.js</code>	Lógica de negocio por dominio
Controladores PDF	<code>src/controllers/reportsController.js</code>	Generación de documentos PDF

6.2 Frontend — Capas de la Aplicación

Capa	Archivo(s)	Responsabilidad
Configuración HTTP	<code>src/api.js</code>	Instancia axios con baseURL y headers
Gestión de Estado Global	<code>src/context/AuthContext.js</code>	Sesión de usuario
Tema Visual	<code>src/ThemeContext.js</code>	Modo oscuro/claro
Estado de Contexto	<code>src/SelectedAnimalContext.js</code>	Animal seleccionado activo
Enrutamiento	<code>src/App.js</code>	Rutas protegidas/públicas, layout
Páginas	<code>src/pages/*.js</code>	Vistas completas por módulo
Componentes	<code>src/components/*.js</code>	Componentes reutilizables
Calculadoras	<code>src/components/calculadoras/*.js</code>	Herramientas clínicas especializadas
Utilidades	<code>src/utils/pdfGenerator.js</code>	Generación de PDFs en cliente

6.3 Principio de Responsabilidad Única (SRP)

Cada módulo tiene **una sola razón para cambiar**:

- `authMiddleware.js` → Solo cambia si cambia la lógica de autenticación
- `errorHandler.js` → Solo cambia si cambia el formato de respuestas de error
- `connection.js` → Solo cambia si cambia el motor de base de datos
- `api.js` → Solo cambia si cambia la URL base del backend
- `ThemeContext.js` → Solo cambia si cambia la lógica de temas
- `SelectedAnimalContext.js` → Solo cambia si cambia la lógica de colores de animales

7. Módulos del Sistema como Objetos de Dominio

Cada ruta del backend representa un **objeto de dominio** con sus operaciones CRUD:

Módulo/Objeto	Ruta API	Operaciones
Clinica	<code>/api/clinicas</code>	registro, login, perfil, cambiar-password
Empleado	<code>/api/empleados</code>	login, CRUD, cambiar-password
Rol	<code>/api/roles</code>	CRUD completo
Tutor	<code>/api/tutores</code>	CRUD, vetar, dar-de-baja (soft delete)
Paciente	<code>/api/pacientes</code>	CRUD, reasignar-tutor
Expediente	<code>/api/expedientes</code>	crear, listar por paciente
Consulta	<code>/api/consultas</code>	crear, listar por expediente, listar todas
Hospitalizacion	<code>/api/hospitalizaciones</code>	crear, listar (con equipo médico)

Módulo/Objeto	Ruta API	Operaciones
Cirugia	/api/cirugias	crear, listar (con anestesia y equipo)
Vacuna	/api/vacunas	crear, listar por paciente, listar todas
Recibo	/api/recibos	CRUD, detalle con items
Inventario	/api/inventario	CRUD, solicitudes de reabastecimiento
Dashboard	/api/dashboard	KPIs por tipo de usuario
Notificacion	/api/notificaciones	listar recordatorios próximos
Calculadora	/api/calculadora	catálogos de medicamentos y toxinas
Reporte	/api/reports	generación de PDFs por tipo

8. Hooks Personalizados como Objetos de Comportamiento

Los hooks de React son equivalentes a **mixins** o **traits** en POO clásica:

```
// useAuth() – Mixin de autenticación
// Inyecta comportamiento de sesión en cualquier componente
const { isLoggedIn, tipo, user, logout } = useAuth();

// useTheme() – Mixin de tema
// Inyecta comportamiento de tema en cualquier componente
const { isDark, toggleTheme } = useTheme();

// useSelectedAnimal() – Mixin de contexto de animal
// Inyecta el animal activo en cualquier componente
const { selectedAnimal, setSelectedAnimal, selectedAnimalColor } =
useSelectedAnimal();
```

Cada hook encapsula:

1. **Estado** (datos)
2. **Comportamiento** (funciones)
3. **Efectos secundarios** (sincronización con localStorage, API, DOM)

9. Inmutabilidad y Funciones Puras

ANA-VET aplica principios de programación funcional complementarios a POO:

```
// Funciones puras en Dashboard.js – sin efectos secundarios
function safeCurrency(val) {
  const n = Number(val ?? 0);
  return isNaN(n) ? '$0.00' : `$$${n.toLocaleString('es-MX', {...})}`;
}
```

```
function safeInt(val) {
  const n = Number(val ?? 0);
  return isNaN(n) ? 0 : n;
}

function getGreeting() {
  const h = new Date().getHours();
  if (h < 12) return 'Buenos días';
  if (h < 19) return 'Buenas tardes';
  return 'Buenas noches';
}

// Función pura de color en SelectedAnimalContext.js
export function stringToColor(value) {
  const str = value?.trim().toLowerCase() || '';
  let hash = 0;
  for (let i = 0; i < str.length; i++) {
    hash = str.charCodeAt(i) + ((hash << 5) - hash);
  }
  return `hsl(${Math.abs(hash % 360)}, 60%, 50%)`;
}
```

10. Resumen de Principios SOLID Aplicados

Principio	Descripción	Aplicación en ANA-VET
S — Single Responsibility	Una clase/módulo, una responsabilidad	Cada archivo de ruta maneja un solo dominio; <code>errorHandler.js</code> solo maneja errores
O — Open/Closed	Abierto para extensión, cerrado para modificación	Nuevas rutas se agregan en <code>index.js</code> sin modificar las existentes; nuevos reportes se agregan en <code>reportsController.js</code>
L — Liskov Substitution	Los subtipos deben ser sustituibles	Los middlewares <code>soloClinica</code> , <code>soloEmpleado</code> , <code>clinicaOVeterinario</code> son intercambiables en las rutas
I — Interface Segregation	Interfaces específicas mejor que una general	<code>useAuth()</code> , <code>useTheme()</code> , <code>useSelectedAnimal()</code> son hooks específicos en lugar de un hook monolítico
D — Dependency Inversion	Depender de abstracciones, no de implementaciones	Los componentes dependen de <code>API</code> (abstracción axios) y de los contextos, no de implementaciones concretas

Documentación generada a partir del código fuente del proyecto ANA-VET · Mayo 2026